

Intermediate Code Generation

1. What is intermediate code?

Ans: During the translation of a source program into the object code for a target machine, a compiler may generate a middle-level language code, which is known as **intermediate code** or **intermediate text**. The complexity of this code lies between the source language code and the object code. The intermediate code can be represented in the form of postfix notation, syntax tree, directed acyclic graph (DAG), three-address code, quadruples, and triples.

2. Write down the benefits of using an intermediate code generation over direct code generation?

Ans: The benefits of using an intermediate code over direct code generation are as follows:

- ❑ Intermediate code is machine independent, which makes it easy to retarget the compiler to generate code for newer and different processors.
- ❑ Intermediate code is nearer to the target machine as compared to the source language so it is easier to generate the object code.
- ❑ The intermediate code allows the machine-independent optimization of the code. Several specialized techniques are used to optimize the intermediate code by the front end of the compiler.
- ❑ Syntax-directed translation implements the intermediate code generation; thus, by augmenting the parser, it can be folded into the parsing.

3. What are the two representations to express intermediate languages?

Ans: The two representations of intermediate languages are categorized as follows:

- ❑ **High-level intermediate representation:** This representation is closer to the source program. Thus, it represents the high-level structure of a program, that is, it depicts the natural hierarchical structure of the source program. The examples of this representation are directed acyclic graphs (DAG) and syntax trees. This representation is suitable for static type checking task. The critical features of high-level representation are given as follows:

- It retains the program structure as it is nearer to the source program.
 - It can be constructed easily from the source program.
 - It is not possible to break the source program to extract the levels of code sharing due to which the code optimization in this representation becomes a bit complex.
- **Low-level intermediate representation:** This representation is closer to the target machine where it represents the low-level structure of a program. It is appropriate for machine-dependent tasks like register allocation and instruction selection. A typical example of this representation is three-address code. The critical features of low-level representation are given as follows:
- It is near to the target machine.
 - It makes easier to generate the object code.
 - High effort is required by the source program to generate the low-level representation.

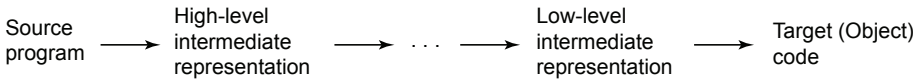


Figure 7.1 A Sequence of Intermediate Representation

4. What is postfix notation? Explain with example.

Ans: Generally, we use infix notation to represent an arithmetic expression such as multiplication of two operands a and b . In infix notation, operator is always placed between the two operands, as $a * b$. But in **postfix notation** (also known as **reverse polish** or **suffix notation**), the operator is shifted to the right end, as $ab*$. In postfix notation, parentheses are not required because the position and the number of arguments of the operator allow only a single way of decoding the postfix expression. The postfix notation can be applied to k -ary operators for any $k > 1$. If β is a k -ary operator and $a_1, a_2, \dots, a_k$ are any postfix expressions, then after applying β to the expressions, the expression in postfix notation is represented as $a_1 a_2 \dots a_k \beta$.

For example, consider the following infix expressions and their corresponding postfix notations:

- $(1 + m) * n$ is an infix expression, the postfix notation will be $1 m + n *$.
- $p * (q + r)$ is an infix expression, the postfix expression will be $p q r + *$.
- $(p - q) * (r + s) + (p - q)$ is an infix expression, the postfix expression will be $p q - r s + * p q - +$.

5. Explain the process of evaluation of postfix expressions.

Ans: The postfix notations can easily be evaluated by using a stack, and generally the evaluation process scans the postfix code left to right.

1. If the scan symbol is an operand, then it is pushed onto the stack, and the scanning is continued.
2. If the scan symbol is a binary operator, then the two topmost operands are popped from the stack. The operator is applied to these operands, and the result is pushed back to the stack.
3. If the scan symbol is a unary operator, it is applied to the top of the stack and the result is pushed back onto the stack.

Note: The result of a unary operator can be shown within parenthesis. For example, $(-X)$.

6. Convert the following expression to the postfix notation and evaluate it.

$P + (-Q + R * S)$

Ans: The postfix notation for the given expression is:

$PQ - RS * ++$

The step-by-step evaluation of this postfix expression is shown in Figure 7.2.

S. no.	String and scan symbol	Previous stack content	Rule in use	New stack content
1.	PQ - RS * ++			
2.	P		Rule 1	P
3.	Q	P	Rule 1	PQ
4.	-	PQ	Rule 3	P (-Q)
5.	R	P (-Q)	Rule 1	P (-Q) R
6.	S	P (-Q) R	Rule 1	P (-Q) RS
7.	*	P (-Q) RS	Rule 2	P (-Q) (R * S)
8.	+	P (-Q) (R * S)	Rule 2	P (-Q + R * S)
9.	+	P (-Q + R * S)	Rule 2	P + (-Q + R * S)

Figure 7.2 Evaluation of Postfix Expression PQ - RS * ++

The desired result is $P + (-Q + R * S)$.

7. What is a three-address code? What are its types? How it is implemented?

Ans: A string of the form $X := Y \text{ OP } Z$, in which op is a binary operator, Y and Z are the addresses of the operands, and X is the address of the result of the operation, is known as **three-address statement**. The operator op can be a fixed or floating-point arithmetic operator, or a logical operator. X , Y , and Z can be considered either as constants or as predefined names by the programmer or temporary names generated by the compiler. This statement is named as the “three-address statement” because of the usage of three addresses, one for the result and two for the operands. The sequence of such three-address statements is known as **three-address code**. The complicated arithmetic expressions are not allowed in three-address code because only a single operation is allowed per statement. For example, consider an expression $A + B * C$, this expression contains more than one operator so the representation of this expression in a single three-address statement is not possible. Hence, the three-address code of the given expression is as follows:

$$\begin{aligned} T_1 &:= B * C \\ T_2 &:= A + T_1 \end{aligned}$$

where, T_1 and T_2 are the temporary names generated by the compiler.

Types of Three-Address Statements:

There are some cases where a statement consists of less than three addresses and is still known as three-address statement. Hence, the different forms of three-address statements are given as follows:

- **Assignment statements:** These statements can be represented in the following forms:
 - $X := Y \text{ op } Z$, where op is any logical/arithmetic binary operator.
 - $X := \text{op } Y$, where op is an unary operator such as logical negation, conversion operators, and shift operators.
 - $X := Y$, where the value of Y is assigned to operand X .
- **Indexed assignment statements:** These statements can be represented in the following forms:
 - $X := Y[I]$
 - $X[I] := Y$, where X , Y and I refer to the data objects and are represented by pointers to the symbol table.

- **Address and pointer assignment statements:** These statements can be represented in the following forms:
 - $X := \text{addr } Y$ defines that X is assigned the address of Y .
 - $X := *Y$ defines that X is assigned the content of location pointed to by Y .
 - $*X := Y$ sets the r-value of the object pointed to by X to the r-value of Y .
- **Jump statements:** Jump statements are of two types—conditional and unconditional that works with relational operators and are represented in the following forms:
 - The unconditional jump is represented as `goto L`, where L being a label. This instruction means that the L^{th} three-address statement is the next to be executed.
 - The conditional jumps such as `if X relop Y goto L`, where `relop` signifies the relational operator ($\leq, =, >$, etc.) applied between X and Y . This instruction implies that if the result of the expression $X \text{ relop } Y$ is true then the statement labeled L is executed. Otherwise, the statement immediately following the `if X relop Y goto L` is executed.
- **Procedure call/return statements:** These statements can be defined in the following forms:
 - `param X` and `call P, n`, where they are represented and typically used in the three-address statement as follows:

```

param X1
param X2
.
.
.
param Xn
call P, n

```

Here, the sequence of three-address statements is generated as a part of call of the procedure $P(X_1, X_2, \dots, X_n)$, and n in `call P, n` is defined as an integer specifying the total number of actual parameters in the call.

- `Y = call p, n` represents the function call.
- `return Y`, represents the return statement, where Y is a returned value.

Implementation of Three-Address Statements:

The three-address statement is an abstract form of intermediate code. Hence, the actual implementation of the three-address statements can be done in the following ways:

- Quadruples
- Triples
- Indirect triples

8. Explain quadruples with the help of a suitable example.

Ans: **Quadruple** is defined as a record structure used to represent a three-address statement. It consists of four fields. The first field contains the operator, the second and third fields contain the operand 1 and operand 2, respectively, and the last field contains the result of that three-address statement. For better understanding of quadruple representation of any statement, consider a statement, $S = -z/a * (x + y)$, where $-z$ stands for unary minus z .

To represent this statement into quadruple representation, we first construct the three-address code as follows:

```

t1 : = x + y
t2 : = a * t1
t3 : = - z
t4 : = t3/t2
S : = t4

```

The quadruple representation of this three-address code is shown in Figure 7.3.

	Operator	Operand 1	Operand 2	Result
0	+	x	Y	t ₁
1	*	a	t ₁	t ₂
2	-	z		t ₃
3	/	t ₃	t ₂	t ₄
4	: =	t ₄		S

Figure 7.3 Quadruple Representation for $S = -z/a * (x + y)$

9. Define triples and indirect triples. Give suitable examples for each.

Ans: Triples: A triple is also defined as a record structure that is used to represent a three-address statement. In triples, for representing any three-address statement three fields are used, namely, operator, operand 1 and operand 2, where operand 1 and operand 2 are pointers to either symbol table or they are pointers to the records (for temporary variables) within the triple representation itself. In this representation, the result field is removed to eliminate the use of temporary names referring to symbol table entries. Instead, we refer the results by their positions. The pointers to the triple structure are represented by parenthesized numbers, whereas the symbol-table pointers are represented by the names themselves. The triples representation of the expression (in Question 7) is shown in Figure 7.4.

	Operator	Operand 1	Operand 2
0	+	x	Y
1	*	a	(0)
2	-	z	
3	/	(2)	(1)
4	: =	S	(3)

Figure 7.4 Triple Representation for $S = -z/a * (x + y)$

In triple representation, the ternary operations $X[I] : = Y$ and $X : = Y[I]$ are represented by using two entries in the triple structure as shown in Figure 7.5(a) and (b) respectively. For the operation $X[I] : = Y$, the names X and I are put in one triple, and Y is put in another triple. Similarly, for the operation $X : = Y[I]$, we can write two instructions, $t : = Y[I]$, and $X : = t$. Note that instead of referring the temporary t by its name, we refer it by its position in the triple.

Indirect triples: An indirect triple representation consists of an additional array that contains the pointers to the triples in the desired order. Let us define an array A that contains pointers to triples in desired order. Indirect triple representation for the statement S given in the previous question is shown in Figure 7.6.

	Operator	Operand 1	Operand 2
0	[] =	X	I
1		Y	

(a) Triple Representation of $X[I] := Y$

	Operator	Operand 1	Operand 2
0	= []	Y	I
1	:=	(0)	X

(b) Triple Representation of $X := Y[I]$

Figure 7.5 More Triple Representations

	A
101	(0)
102	(1)
103	(2)
104	(3)
105	(4)

	Operator	Operand 1	Operand 2
0	+	x	y
1	*	a	(0)
2	-	z	
3	/	(2)	(1)
4	:=	S	(3)

Figure 7.6 Indirect Triples Representation of $S = -z/a * (x + y)$

The main advantage of indirect triple representation is that an optimizing compiler can move an instruction by simply reordering the array A, without affecting the triples themselves.

10. Explain Boolean expressions. What are the different methods available to translate Boolean expression?

Ans: Boolean operators, like AND(&&), OR(||) and NOT(!), play an important role in constructing the Boolean expressions. These operators are applied to either relational expressions or Boolean variables. In programming languages, Boolean expressions serve two main functions, given as follows:

- Boolean expressions can be used as conditional expressions in the statements that alter the flow of control, such as in while- or if-then-else statements.
- Boolean expressions are also used to compute the logical values.

Boolean expressions can be generated by using the following grammar:

$$S \rightarrow S \text{ or } S \mid S \text{ and } S \mid \text{not } S \mid (S) \mid \text{id} \mid \text{id rel op id} \mid \text{true} \mid \text{false}$$

where the attribute `rel op` is used to indicate any of $<, \leq, =, \neq, >, \geq$. Generally, we consider that the operators **or** and **and** are left-associative, and that the operator **not** has the highest precedence, then **and**, and then **or**.

Methods of Translating a Boolean Expression into Three-Address Code:

There are two methods available to translate a Boolean expression into three-address code, as given below:

- **Numerical representation:** The first method of translating Boolean expression into three-address code comprises encoding true and false numerically and then evaluating the Boolean expression similar to an arithmetic expression. True is often denoted by 1 and false by 0. Some other encodings are also possible where any non-zero or non-negative quantity indicates true and any negative

or zero number indicates false. Expressions will be calculated from left to right like arithmetic expressions. For example, consider a Boolean expression X and Y or Z , the translation of this expression into three-address code is as follows:

```
t1: = X and Y
t2: = t1 or Z
```

Now, consider a relational expression `if  $X > Y$  then 1 else 0`, the three-address code translation for this expression is as follows:

```
1. if X > Y goto (4)
2. t1: = 0
3. goto (5)
4. t1: = 1
5. Next
```

Here, t_1 is a temporary variable that can have the value 1 or 0 depending on whether the condition is evaluated to true or false. The label `Next` represents the statement immediately following the `else` part.

- **Control-flow representation:** In the second method, the Boolean expression is translated into three-address code based on the flow of control. In this method, the value of a Boolean expression is represented by a position reached in a program. In case of evaluating the Boolean expressions by their positions in program, we can avoid calculating the entire expression. For example, if a Boolean expression is X and Y , and if X is false, then we can conclude that the entire expression is false without having to evaluate Y . This method is useful in implementing the Boolean expressions in control-flow statements such as `if-then-else` and `while-do` statements. For example, we consider the Boolean expressions in context of conditional statements such as

- If X then S_1 else S_2
- while X do S

In the first statement, if X is true, the control jumps to the first statement of the code for S_1 , and if X is false, the control jumps to the first statement of S_2 as shown in Figure 7.7(a). In case of second statement, when X is false, the control jumps to the statement immediately following the `while` statement, and if X is true, the control jumps to the first statement of the code for S as shown in Figure 7.7(b).

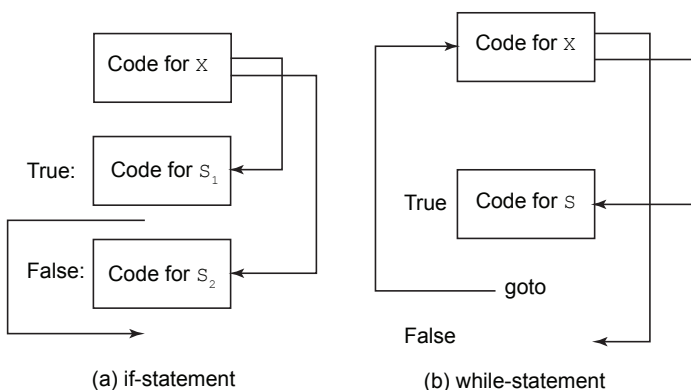


Figure 7.7 Control-flow Translation of Boolean Expressions

11. What is postfix translation?

Ans: A translation scheme is said to be postfix translation if, for each production $S \rightarrow \alpha$, the transition rule for $S.CODE$ consists of the concatenation of the $CODE$ translations of the non-terminals in α , in the same order as the non-terminals appear in α , followed by a tail of output. It is easy to use the postfix translation of $CODE$ as it reduce space requirements, otherwise we have to follow a long scheme to construct the intermediate language form of any program like generating a parse tree followed by a walk of the tree.

12. Explain the array references in arithmetic expressions.

Or

Discuss the addressing of array elements.

Ans: An **array** is a collection of items having similar data type, which are stored in a block of consecutive memory locations. In case of languages like C and Java, array consists of n elements, numbered $0, 1, \dots, n - 1$. For a single-dimensional array, the address of i^{th} element of the array is calculated as $base + i * w$, where $base$ is the relative address of array element $X[0]$, and w is the width of each array element. In case of a two-dimensional array, the relative address of the array element $X[i_1][i_2]$ is calculated as $base + i_1 * w_1 + i_2 * w_2$ where w_1 is the width of a row and w_2 is the width of an element in a row. In general, for a k -dimensional array, the formula can be written as follows:

$$base + i_1 * w_1 + i_2 * w_2 + \dots + i_k * w_k \quad (1)$$

We can also determine the relative address of an array element in terms of the number of elements n_k along k dimensions of the array with each element of width w . In this case, the address calculations are done on the basis of row-major or column-major layout of the arrays. Consider a two-dimensional array $X[2][3]$, which can be stored either in a row-major form or in a column-major form as shown in Figure 7.8.

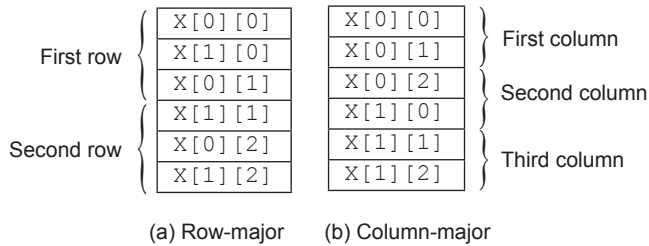


Figure 7.8 Layouts of a Two-dimensional Array

If the elements of a two-dimensional array $X[n_1][n_2]$ are stored in a row-major form, the relative address of an array element $X[i_1][i_2]$ is calculated as follows:

$$base + (i_1 * n_2 + i_2) * w$$

On the other hand, if the elements are stored in a column-major form, the relative address of $X[i_1][i_2]$ is calculated as follows:

$$base + (i_1 + i_2 * n_1) * w$$

The row-major and column-major forms can be generalized to k -dimensions. If we generalize row-major form, then elements are stored in such a way that when we scan down a block of storage, the rightmost scripts appear to vary fastest. On the other hand, in case of column-major form, the leftmost scripts appear to vary fastest.

In general, the array elements in one-dimensional array need not be numbered as $0, 1, \dots, n - 1$, rather they can be numbered as $low, low + 1, \dots, high$. Now, the address of an array reference $X[i]$ can be calculated as $base + (i - low) * w$. Here, $base$ is the relative address of $A[low]$.

13. Explain the translation of array references.

Ans: The main problem while translating and generating the intermediate code for an array references is to relate the address calculation formulas to a grammar for array references. Consider the following grammar, where the non-terminal M generates an array name followed by a sequence of index expressions:

$$M \rightarrow M[A] \mid \text{digit}[A]$$

Figure 7.9 shows the translation scheme that generates three-address code for expressions with array references. It comprises the productions and semantic routines for generating three-address code for expressions incrementally. In addition, it comprises the productions involving the non-terminal M . We have also assumed that the addresses are calculated using the formula (1), which is based on the width of the array elements.

$S \rightarrow \text{digit} = A$	$\{ \text{gen } (\text{top.get}(\text{digit.lexeme}))' = 'A.\text{addr} \}; \}$
$ M = A$	$\{ \text{gen } (M.\text{addr}.\text{base}' ['M.\text{addr}']') = 'E.\text{addr} \}; \}$
$A \rightarrow A_1 + A_2$	$\{ A.\text{addr} = \text{newTemp}(); \}$ $\text{gen } (A.\text{addr}' = 'A_1.\text{addr}' + 'A_2.\text{addr} \}; \}$
$ \text{digit}$	$\{ A.\text{addr} = \text{top.get}(\text{digit.lexeme}); \}$
$ M$	$\{ A.\text{addr} = \text{newTemp}(); \}$ $\text{gen } (A.\text{addr}' = 'M.\text{array}.\text{base}' ['M.\text{addr}']') \}; \}$
$M \rightarrow \text{digit}[A]$	$\{ M.\text{array} = \text{top.get}(\text{digit.lexeme}); \}$ $M.\text{type} = M.\text{array}.\text{type}.\text{elem};$ $M.\text{addr} = \text{newTemp}();$ $\text{gen } (M.\text{addr}' = 'A.\text{addr}' * 'M.\text{type}.\text{width} \); \}$
$ M_1[A]$	$\{ M.\text{array} = M_1.\text{array}; \}$ $M.\text{type} = M_1.\text{type}.\text{elem};$ $t = \text{newTemp}();$ $M.\text{addr} = \text{newTemp}();$ $\text{gen } (t' = 'A.\text{addr}' * 'M.\text{type}.\text{width} \); \}$ $\text{gen } (M.\text{addr}' = 'M_1.\text{addr}' + 't \); \}$

Figure 7.9 Semantic Actions for Array References

In Figure 7.9, the non-terminal M has three synthesized attributes $M.\text{addr}$, $M.\text{array}$, and $M.\text{type}$. Here $M.\text{type}$ represents a temporary to be used during computation of the offsets of $i_j * w_j$ terms, $M.\text{array}$ denotes a pointer pointing to the symbol table entry for an array name, and $M.\text{type}$ is the type of the subarray generated by M .

In the semantic actions of Figure 7.9, the first production $S \rightarrow \text{digit} = A$ shows an assignment to a non-array variable. In the second production $S \rightarrow M = A$, an index copy instruction is generated by the semantic action to assign the value of expression A to the location of array reference M . As discussed earlier, the symbol table entry for the array is obtained by the attribute $M.\text{array}$. The attribute $M.\text{array}.\text{base}$ gives the base address of the array which is the address of its 0^{th} element and the attribute $M.\text{addr}$

is a temporary variable that holds the offset for array reference generated by M . Thus, $M.array.base[M.addr]$ gives the location for the array reference. The r-value from address $A.addr$ is copied into the M 's location by the generated instruction.

The production $A \rightarrow M$ has a semantic rule associated with it that generates a code which copies value at location M into a temporary variable $M.array.base[M.addr]$.

14. What do you understand by procedure calls?

Ans: In high-level languages, we use procedures to break down a large program into smaller modular components, and the procedures that return some value are called **functions**. The procedure or function is an important and frequently used programming construct. A compiler is expected to generate a good three-address code for procedure calls and returns. The runtime routine that handles procedure argument passing, procedure calls and returns, is a part of runtime support package. For example, consider the following grammar for a simple procedure call statement:

```
S → call id(Alist)
Alist → Alist, A
Alist → A
```

The translation of a simple procedure call consists of a calling sequence, which is defined as a sequence of actions performed while entering into and exiting from each procedure.

Calling sequence: On the occurrence of a procedure call, space for the activation record of the called procedure activation record is allocated on the stack. The called procedure arguments are evaluated and are made available to the called procedure by storing them at a known location. The environment pointers must be established so that the called procedure can access data in the enclosing blocks. When a call occurs, the state of the procedure is saved so that the execution of the calling procedure can be resumed after the call. The return address, which is the location to which the called procedure must return after its execution, is also saved at a known location. Finally, a jump to the first statement of the code for the called procedure must be generated.

Return sequence: When the control reaches the last statement of the calling procedure, that is, when the procedure returns, several actions need to be performed. If a procedure call is a function, then the return value (result) must be saved at some known location. The activation record of the calling procedure is then restored and finally a jump to the return address of calling procedure is generated.

Translation of Procedure Calls:

1. $S \rightarrow \text{call id}(Alist),$
 {for each item R on queue do
 emit ('param ' R);
 emit ('call'id.place)}

The code for S is the code for $Alist$, which evaluates the arguments, followed by the param R statement for each argument, followed by a call statement.

2. $Alist \rightarrow Alist, A$
 {append $A.place$ to the end of queue}
3. $Alist \rightarrow A$
 {initialize queue to contain only $A.place$ }

At this point, the queue is cleared and gets a single pointer to the symbol table location for the name that denotes the value of A .

15. Explain declarations with the help of a suitable example.

Ans: The general form of declarations in any programming language is a keyword (that denotes an attribute) followed by a list of names having that attribute. The semantic routine/action associated with each declaration statement should enter the declared attribute into the symbol-table entry for each name on the list. The productions for such declarations are given below:

```
S → double namelist
    | float namelist
namelist → id, namelist
         | id
```

Here, the declarations within productions are in such a form which would raise a problem that if someone wants to get at the attribute associated with the `namelist`, then he has to wait until the entire list of `id`'s has been reduced to `namelist`. Thus, when `id` or `id, namelist` is reduced to `namelist`, the SDT schemes based on these productions cannot create the correct symbol table entries. To avoid this kind of difficulty, we can follow any of these approaches:

- In the first approach, we can create a separate list for `double` and `float` as given below:

```
S → double doublelist
    | float floatlist
doublelist → id, doublelist
           | id
floatlist → id, floatlist
          | id
```

This approach is based on the assumption that the LR parser would be able to decide whether to reduce the first `id` to `doublelist` or `floatlist`. This approach is not desirable for large numbers of attributes because as the number of attributes increases, the number of productions also increases. This would create a decision problem for the parser.

- In the second approach, we can simply rewrite the grammar rules by considering the translation of names just as a list of names. Now, the above rules can be rewritten as follows:

```
S → S, id
    | double id
    | float id
```

Now, we can define the semantic actions for these rules as follows:

```
S → double id {Enter(id.place, double); S.attr: = double}
S → float id  {Enter(id.place, float); S.attr: = float}
S → S1, id   {Enter(id.place, S1.attr);
               S.attr: = S1.attr}
```

These semantic actions can now enter the appropriate attribute into the symbol table for each name on the list. Here, `S.attr` is the translation attribute for non-terminal `S`, the procedure `enter(p, x)` associates attribute `x` to the symbol table entry pointed to by `p`, and `id.place` points the symbol table entry for the name represented by the token `id`.

16. Discuss the syntax directed translation of case statements.

Ans: The syntax for `case` or `switch` statement is

```

switch E
begin
case  $V_1$ :  $S_1$ 
    case  $V_2$ :  $S_2$ 
    .
    .
    .
    case  $V_{n-1}$ :  $S_{n-1}$ 
    default:  $S_n$ 
end

```

where

E is an expression to be evaluated;

$V_1, V_2, \dots, V_{n-1}$ are the distinct values and are known as **case labels**, and V_n is a default statement;

$S_1, S_2, \dots, S_{n-1}$ are the statements that will be executed when a particular value is matched.

The case values are constants and are selected by the selector expression E . First, E is evaluated and the resultant value is matched with these constant values. Then the associated statement sequence of the matched case value is executed. There is a default expression which is always executed if no other value is matched.

Syntax directed translation of case statements: A simple syntax directed translation scheme translates the case statements into an intermediate code as shown in Figure 7.10.

```

                                code to evaluate E into t
                                goto SAMPLE
A1:    code for S1
                                goto REF
A2:    code for S2
                                goto REF
    .
    .
    .
An-1:  code for Sn-1
                                goto REF
An:    code for Sn
                                goto REF
SAMPLE: if t = v1 goto A1
        if t = v2 goto A2
        if t = v3 goto A3
    .
    .
    .
        if t = Vn-1 goto An-1
        goto An
REF:

```

Figure 7.10 Translation of Case Statement into Intermediate Code

When the switch keyword is encountered, two labels SAMPLE and REF, and a temporary variable t are generated. As we start parsing, we find the expression E and now we generate a code to evaluate this expression in the temporary t . When E is processed, we generate the jump goto REF.

On the occurrence of each `case` keyword, a new label A_i is created and entered into the symbol table. The cases are stored on a stack, which contains a pointer to this symbol entry along with the value V_i of each case constant. The evaluated expression in temporary t is matched with the available values $V_1, V_2, \dots, V_{n-1}$ and if a value match occurs then the corresponding statements are executed. If no value is matched, then the default case A_n is executed.

Note that all the test expressions appear at the end. This enables a simple code generator to recognize the multiway branch and to generate an efficient code for it. If the branching conditions are placed at the beginning then the compiler would have to perform extensive analysis to generate the most efficient implementation.

17. What is backpatching? Explain.

Ans: The syntax directed definitions can be easily implemented by using two passes. In the first pass, we construct a syntax tree for the input, and in the second pass, we traverse the tree in depth first order to complete the translations in the given definition. Generating code for flow of control statements and Boolean expressions is difficult in single pass. This is because we may not be able to know the labels that the control must goto during the generation of jump statements. Thus, the generated code would be a series of branching statements in which the targets of the jumps are temporarily left unspecified. To overcome this problem, we use back patching, which is a technique to solve the problem of replacing symbolic names into the `goto` statements by the actual target addresses. However, some languages do not permit to use symbolic names in the branches, for this we maintain a list of branches that have the same target labels and then replace them once they are defined. To manipulate the lists of jumps, the following three functions are used:

- **makelist(i)** : This function creates a new list containing an index i into the array of statements and then returns a pointer pointing to the newly created list.
- **merge(p_1, p_2)** : This function concatenates the two lists pointed to by the pointers p_1 and p_2 respectively, and then returns a pointer to the concatenated list.
- **backpatch(p, i)** : This function inserts i as the target labels for each of the statements on the list pointed to by p .

18. Translate the expression $a := -b * (c + d) / e$ into quadruples and triple representation.

Ans: The three-address code for the given expression is given below:

```

t1 := -b (here, '-' represents unary minus)
t2 := c + d
t3 := t1 * t2
t4 := t3 / e
a := t4

```

The quadruple representation of this three-address code is shown in Figure 7.11.

	Operator	Operand 1	Operand 2	Result
0	-	b		t ₁
1	+	c	d	t ₂
2	*	t ₁	t ₂	t ₃
3	/	t ₃	e	t ₄
4	=	t ₄		a

Figure 7.11 Quadruple Representation for $a := -b * (c + d) / e$

The triple representation for the expression is given in Figure 7.12.

	Operator	Operand 1	Operand 2
0	-	b	
1	+	c	d
2	*	(0)	(1)
3	/	(2)	e
4	=	a	(3)

Figure 7.12 Triple Representation for $a = -b * (c + d)/e$

19. Translate the expression $X = -(a + b) * (c + d) + (a + b + c)$ into quadruples and triples.

Ans: The three-address code for the given expression is given below:

```

t1 := a + b
t2 := -t1
t3 := c + d
t4 := t2 * t3
t5 := t1 + c
t6 := t4 + t5
X := t6

```

The quadruple representation is shown in Figure 7.13.

	Operator	Operand 1	Operand 2	Result
0	+	a	b	t ₁
1	-	t ₁		t ₂
2	+	c	d	t ₃
3	*	t ₂	t ₃	t ₄
4	+	t ₁	c	t ₅
5	+	t ₄	t ₅	t ₆
6	=	t ₆		

Figure 7.13 Quadruple Representation for $X = -(a + b) * (c + d) + (a + b + c)$

The triple representation for the given expression is shown in Figure 7.14.

	Operator	Operand 1	Operand 2
0	+	a	b
1	-	(0)	
2	+	c	d
3	*	(1)	(2)
4	+	(0)	c
5	+	(3)	(4)
6	=	(5)	

Figure 7.14 Triple Representation for $X = -(a + b) * (c + d) + (a + b + c)$

20. Generate the three-address code for the following program segment.

```

main()
{
    int k = 1;
    int a[5];
    while (k <= 5)
    {
        a[k] = 0;
        k++;
    }
}

```

Ans: The three-address code for the given program segment is given below:

1. $k := 1$
2. if $k \leq 5$ goto (4)
3. goto (8)
4. $t_1 := k * \text{width}$
5. $t_2 := \text{addr}(a) - \text{width}$
6. $t_2[t_1] := 0$
7. $t_3 := k + 1$
8. $k := t_3$
9. goto (2)
10. Next

21. Generate the three-address code for the following program segment

```

while(x < z and y > s) do
    if x = 1 then
        z = z + 1
    else
        while x <= s do
            x = x + 10;

```

Ans: The three-address code for the given program segment is given below:

1. if $x < z$ goto (3)
2. goto (16)
3. if $y > s$ goto (5)
4. goto (16)
5. if $x = 1$ goto (7)
6. goto (10)
7. $t_1 := z + 1$
8. $z := t_1$
9. goto (1)

```

10. if x <= s goto (12)
11. goto (1)
12.  $t_2 := x + 10$ 
13.  $x := t_2$ 
14. goto (10)
15. goto (1)
16. Next

```

22. Consider the following code segment and generate the three-address code for it.

```

for (k = 1; k <= 12; k++)
    if x < y then a = b + c;

```

Ans: The three-address code for the given program segment is given below:

```

1. k := 1
2. if k <= 12 goto (4)
3. goto (11)
4. if x < y goto (6)
5. goto (8)
6.  $t_1 := b + c$ 
7.  $a := t_1$ 
8.  $t_2 := k + 1$ 
9.  $k := t_2$ 
10. goto (2)
11. Next

```

23. Translate the following statement, which alters the flow of control of expressions, and generate the three-address code for it.

```

while (P < Q) do
    if (R < S) then a = b + c;

```

Ans: The three-address code for the given statement is as follows:

```

1. if P < Q goto (3)
2. goto (8)
3. if R < S goto (5)
4. goto (1)
5.  $t_1 := b + c$ 
6.  $a := t_1$ 
7. goto (1)
8. Next

```

24. Generate the three-address code for the following program segment where, x and y are arrays of size $10 * 10$, and there are 4 bytes/word.

```
begin
  add = 0
  a = 1
  b = 1
  do
    begin
      add = add + x[a,b] * y[a,b]
      a = a + 1
      b = b + 1
    end
  while a <= 10 and b <= 10
end
```

Ans: The three-address code for the given program segment is given below:

1. add := 0
2. a := 1
3. b := 1
4. $t_1 := a * 10$
5. $t_1 := t_1 + b$
6. $t_1 := t_1 * 4$
7. $t_2 := \text{addr}(x) - 44$
8. $t_3 := t_2[t_1]$
9. $t_4 := b * 10$
10. $t_4 := t_4 + a$
11. $t_4 := t_4 * 4$
12. $t_5 := \text{addr}(y) - 44$
13. $t_6 := t_5[t_4]$
14. $t_7 := t_3 * t_6$
15. $t_7 := \text{add} + t_7$
16. $t_8 := a + 1$
17. a := t_8
18. $t_9 := b + 1$
19. b := t_9
20. if a <= 10 goto (22)
21. goto (23)
22. if b <= 10 goto (4)
23. Next

25. Translate the following program segment into three-address statements:

```
switch(a + b)
{
    case 2: {x = y; break;}
    case 5: switch x
        {
            case 0: {a = b + 1; break;}
            case 1: {a = b + 3; break;}
            default: {a = 2; break;}
        }
    break;
    case 9: {x = y - 1; break;}
    default: {a = 2; break;}
}
```

Ans: The three-address code for the given program segment is given below:

1. $t_1 = a + b$
2. goto (23)
3. $x = y$
4. goto (27)
5. goto (14)
6. $t_3 = b + 1$
7. $a = t_3$
8. goto (27)
9. $t_4 = b + 3$
10. $a = t_4$
11. goto (27)
12. $a = 2$
13. goto (27)
14. if $x = 0$ goto (6)
15. if $x = 1$ goto (9)
16. goto (12)
17. goto (27)
18. $t_5 = y - 1$
19. $a = t_5$
20. goto (27)
21. $a = 2$
22. goto (27)
23. if $t = 2$ goto (3)

```

24. if t = 5 goto (5)
25. if t = 9 goto (18)
26. goto (21)
27. Next

```

Multiple-Choice Questions

- Which of the following is not true for the intermediate code?
 - It can be represented as postfix notation.
 - It can be represented as syntax tree, and or a DAG.
 - It can be represented as target code.
 - It can be represented as three-address code, quadruples, and triples.
- Which of the following is true for intermediate code generation?
 - It is machine dependent.
 - It is nearer to the target machine.
 - Both (a) and (b)
 - None of these
- Which of the following is true in the context of high-level representation of intermediate languages?
 - It is suitable for static type checking.
 - It does not depict the natural hierarchical structure of the source program.
 - It is nearer to the target program.
 - All of these
- Which of the following is true for the low-level representation of intermediate languages?
 - It requires very few efforts by the source program to generate the low-level representation.
 - It is appropriate for machine-dependent tasks like register allocation and instruction selection.
 - It does not depict the natural hierarchical structure of the source program.
 - All of these
- The reverse polish notation or suffix notation is also known as _____.
 - Infix notation
 - Prefix notation
 - Postfix notation
 - None of above
- In a two-dimensional array $A[i][j]$, where i is a element of width w_1 and j is of width w_2 , the relative address of $A[i][j]$ can be calculated by the formula _____.
 - $i * w_1 + j * w_2$
 - $base + i * w_1 + j * w_2$
 - $base + i * w_2 + j * w_1$
 - $base + (i + j) * (w_1 + w_2)$

Answers

1. (c) 2. (c) 3. (a) 4. (b) 5. (c) 6. (b)

Type Checking

1. What is a type system? List the major functions performed by the type systems.

Ans: A **type system** is a tractable syntactic framework to categorize different phrases according to their behaviors and the kind of values they compute. It uses logical rules to understand the behavior of a program and associates types with each compound value and then it tries to prove that no type errors can occur by analyzing the flow of these values. A type system attempts to guarantee that only value-specific operations (that can match with the type of value used) are performed on the values. For example, the floating-point numbers in C uses floating-point specific operations to be performed over these numbers such as floating-point addition, subtraction, multiplication, etc.

The language design principle ensures that every expression must have a type that is known (at the latest, at run time) and a type system has a set of rules for associating a type to an expression. Type system allows one to determine whether the operators in an expression are appropriately used or not. An implementation of type system is called **type checker**.

There are two type systems, namely, *basic type system* and *constructed type system*.

- ❑ **Basic type system:** Basic type system contains atomic types and has no internal structure. They contain integer, real, character, and Boolean. However, in some languages like Pascal, they can have subranges like 1 . . . 10 and enumeration types like orange, green, yellow, amber, etc.
- ❑ **Constructed type system:** Constructed type system contains arrays, records, sets, and structure types constructed from basic types and/or from other constructed types. They also include pointers and functions.

Type system provides some functions that include:

- ❑ **Safety:** A type system allows a compiler to detect meaningless or invalid code which does not make a sense; by doing this it offers more strong typing safety. For example, an expression `5 / "Hi John"` is treated as invalid because arithmetic rules do not specify how to divide an integer by a string.
- ❑ **Optimization:** For optimization, a type system can use static and/or dynamic type checking, where static type checking provides useful compile-time information and dynamic type checking verifies and enforces the constraints at runtime.
- ❑ **Documentation:** The more expressive type systems can use types as a form of documentation to show the intention of the programmer.

- ❑ **Abstraction (or modularity):** Types can help programmers to consider programs as a higher level of representation than bit or byte by hiding lower level implementation.

2. Define type checking. Also explain the rules used for type checking.

Ans: Type checking is a process of verifying the type correctness of the input program by using logical rules to check the behavior of a program either at compile time or at runtime. It allows the programmers to limit the types that can be used for semantic aspects of compilation. It assigns types to values and also verifies whether these values are consistent with their definition in the program.

Type checking can also be used for detecting errors in programs. Though errors can be checked dynamically (at runtime) if the target program contains both the type of an element and its value, but a sound type system eliminates the need for dynamic checking for type errors by ensuring that these errors would not arise when the target program runs.

If the rules for type checking are applied strongly (that is, allowing only those automatic type conversions which do not result in loss of information), then the implementation of the language is said to be *strongly typed*; otherwise, it is said to be *weakly typed*. A strongly typed language implementation guarantees that the target program will run without any type errors.

Rules for type checking: Type checking uses syntax-directed definitions to compute the type of the derived object from the types of its syntactical components. It can be in two forms, namely, *type synthesis* and *type inference*.

- ❑ **Type synthesis:** Type synthesis is used to build the type of an expression from the types of its sub-expressions. For type synthesis, the names must be declared before they are used. For example, the type of expression $E_1 * E_2$ depends on the types of its sub-expressions E_1 and E_2 . A typical rule is used to perform type synthesis and has the following form:

if expression f has a type $s \rightarrow t$ **and** expression x has a type s ,
then expression $f(x)$ will be of type t

Here, $s \rightarrow t$ represents a function from s to t . This rule can be applied to all functions with one or more arguments. This rule considers the expression $E_1 * E_2$ as a function, $\text{mul}(E_1, E_2)$ and uses E_1 and E_2 to build the type of $E_1 * E_2$.

- ❑ **Type inference:** Type inference is the analysis of a program to determine the types of some or all of the expressions from the way they are used. For example,

```
public int mul(int E1, int E2)
    return E1 * E2;
```

Here, E_1 and E_2 are defined as integers. So by type inference, we just need definition of E_1 and E_2 . Since the resulting expression $E_1 * E_2$ uses $*$ operation, which would be taken as integer because it is performed on two integers E_1 and E_2 . Therefore, the return type of `mul` must be an integer. A typical rule is used to perform type inference and has the following form:

if $f(x)$ is an expression,
then for some type variables α and β , f is of type $\alpha \rightarrow \beta$ **and**
 x is of type α

3. Explain type expressions.

Ans: Type expressions are used to represent structure of types and can be considered as a textual representation for types. A type expression can either be of basic type or can be created by applying an operator (known as **type constructor**) to a type expression.

For example, a type expression for the array type `int[3][5]` considers it as an “array of 3 arrays having 5 integers in each of them”, and its type expression can be written as “array (3, array (5, integer))”. The type expression uses a tree to represent the type structure. The tree representation of array type `int[3][5]` is shown in Figure 8.1, where an operator array takes two arguments: a number and a type.

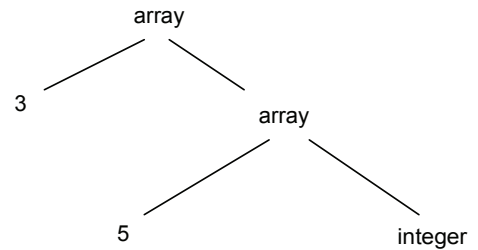


Figure 8.1 Type Expression of `Int[3][5]`

Type expressions can be defined as follows:

- **Basic types:** Every basic types such as Boolean, char, integer, float, void, etc., is a type expression.
- **Type names:** Every type name is a type expression.
- **Constructed types:** A constructed type applies constructors to the type expressions, which can be:
 - **Arrays:** A type expression can be constructed by applying an array type constructor to a number and a type expression. It can be represented as $\text{array}(I, T)$, where I is an index type and T is the type of array elements. For example, $\text{array}(1 \dots 10, \text{integer})$.
 - **Cartesian product:** For any type expressions T_1 and T_2 , the Cartesian product $T_1 \times T_2$ is also a type expression.
 - **Record with field names:** A record type constructor is applied to the field names to form a type expression. For example, $\text{record}\{\text{float } a, \text{ int } b\} \rightarrow X$;
 - **Function types:** A type constructor $\rightarrow$ is used to form a type expression for function types. For example, $A \rightarrow B$ denotes a function from type A to type B . For example, $\text{real} \times \text{real} \rightarrow \text{real}$.
 - **Pointers:** A type expression pointer (T_1) represents a pointer to an object of type T_1 .

4. Define these terms: static type checking, dynamic type checking, and strong typing.

Ans: Static type checking: In static type checking, most of the properties are verified at compile time before the execution of the program. The languages C, C++, Pascal, Java, Ada, FORTRAN, and many more allow static type checking. It is preferred because of the following reasons:

- As the compiler uses type declarations and determines all types at compile time, hence catches most of the common errors at compile time.
- The execution of output program becomes fast because it does not require any type checking during execution.

The main disadvantage of this method is that it does not provide flexibility to perform type conversions at runtime. Moreover, the static type checking is conservative, that is, it will reject some programs that may behave properly at runtime, but that cannot be statically determined to be well-typed.

Dynamic type checking: Dynamic type checking is performed during the execution of the program and it checks the type at runtime before the operations are performed on data. Some languages that support dynamic type checking are Lisp, Java Script, Smalltalk, PHP, etc. Some advantages of the dynamic type checking are as follows:

- It can determine the type of any data at runtime.
- It gives some freedom to the programmer as it is less concerned about types.
- In dynamic typing, the variables do not have any types associated with them, that is, they can refer to a value of any type at runtime.

- ❑ It checks the values of all the data types during execution which results in more robust code.
- ❑ It is more flexible and can support union types, where the user can convert one type to another at runtime.

The main disadvantage of this method is that it makes the execution of the program slower by performing repeated type checks.

Strong typing: A type checking which guarantees that no type errors can occur at runtime is called **strong type checking** and the system is called **strongly typed**. The strong typing has certain disadvantages such as:

- ❑ There are some checks like array bounds checking which require dynamic checking.
- ❑ It can result into performance degradation.
- ❑ Generally, these type systems have holes in the type systems, for example, variant records in Pascal.

5. Write down the process to design a type checker.

Ans: A type checker is an implementation of a type system. The process to design a type checker includes the following steps:

1. **Identifying the available types in the language:** We have two types that are available in the language.
 - Base types (integer, double, Boolean, string, and so on)
 - Compound types (arrays, classes, interfaces, and so on)
2. **Identifying the language constructs with associated types:** Each programming language consists of some constructs and each of them is associated with a type as discussed below:
 - **Constants:** Every constant has an associated type. A scanner identifies the types and associated lexemes of a constant.
 - **Variables:** A variable can be global, local, or an instance of a class. Each of these variables must have a declared type, which can either be one of the base types or the supported compound types.
 - **Functions:** The functions have a return type, and the formal parameters in function definition as well as the actual arguments in the function call also have a type associated with them.
 - **Expressions:** An expression can contain a constant, variable, functional call, or some other operators (like unary or binary operators) that can be applied in an expression. Hence, the type of expression depends on the type of constant, variable, operands, function return type, and on the type of operators.
3. **Identifying the language semantic rules:** The production rules to parse variable declarations can be written as:

```
Variable Declaration → Variable
Variable → Type identifier
Type → int|double|Boolean|string|identifier|Type[]
```

The parser stores the name of an identifier lexeme as an attribute attached to the token. The name associated with the identifier symbol, and the type associated with the identifier and type symbol are used to reduce the variable production. A new variable declaration is created by declaring an identifier of that type and that variable is stored in the symbol table for further lookup.

6. What is type equivalence?

Ans: Type equivalence is used by the type checking to check whether the two type expressions are equivalent or not. It can be done by checking the equivalence between the two types. The rule used for type checking works as follows:

```
if two type expressions are equal then return a certain type else
return a type error()
```

When two type expressions are equivalent, we need a precise definition of both the expressions. When names are given to type expressions, and these names are further used in subsequent type expressions, it may result in potential ambiguities. The key issue is whether a name in a type expression stands for itself, or it is an abbreviation for another type expression.

There are two schemes to check type equivalence of expressions:

Structural equivalence: Structural equivalence needs a graph to represent the type expressions. The two type expressions are said to be structurally equivalent if and only if they hold any of the following conditions:

- They are of identical basic type.
- Same type constructor has been applied to equivalent types to construct the type expressions.
- A type name of one represents the other.

Name equivalence: If type names are treated as standing for themselves, then the first two conditions of structural equivalence lead to another equivalence of type expressions called name equivalence. In other words, name equivalence considers types to be equal if and only if the same type names are used and one of the first two conditions of structure equivalence holds.

For example, consider the following few types and variable declarations.

```
typedef double Value
...
...
Value var1, var2
Sum var3, var4
```

In these statements, `var1` and `var2` are name equivalent, so are `var3` and `var4`, because their type names are same. However, `var1` and `var3` are not name equivalent, because their type names are different.

7. Explain type conversion.

Ans: Type conversion refers to the conversion of a certain type into another by using some semantic rules. Consider an expression `a + b`, where `a` is of type `int` and `b` is of `float`. The representations of floats and integers are different within a computer, and an operation on integers and floats uses different machine instructions. Now, the primary task of the compiler is to convert one of the operands of `+` to make both of the operands to same type. For example, an expression `5 * 7.14` has two types, one is of `float` type and other one is of type `int`. To convert integer type constant into `float` type, we use a unary operator (`float`) as shown here:

```
x = (float) 5
y = x * 7.14
```

The type conversion can be done implicitly or explicitly. The conversion from one type to another is called **implicit**, if it is automatically done by the computer. Usually, implicit conversions of constants

can be done at compile time and it results in an improvement in the execution time of the object program. Implicit type conversion is also known as **coercion**. A conversion is said to be **explicit** if the programmer must write something to cause the conversion. For example, all conversions in Ada are explicit. Explicit conversions can be considered as a function applications to a type checker. Explicit conversion is also known as **casts**.

Conversion in languages can be considered as *widening conversions* and *narrowing conversions*, as shown in Figure 8.2(a) and (b), respectively.

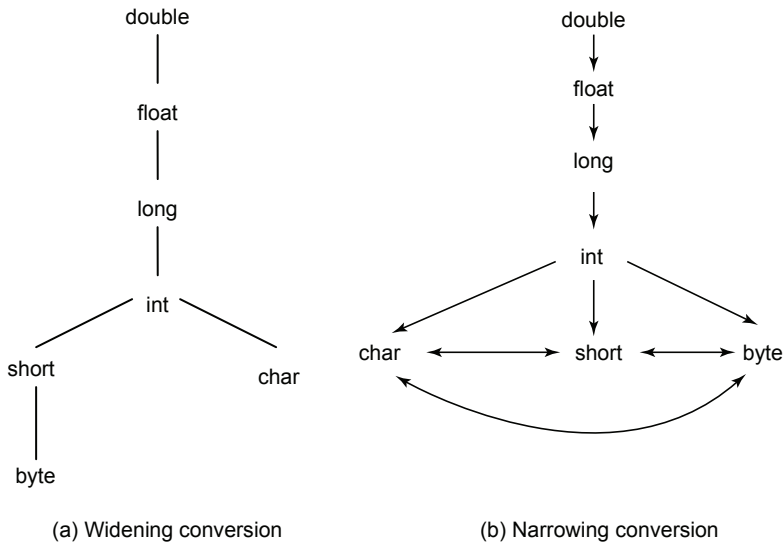


Figure 8.2 Conversion Between Primitive Types in Java

The rules used for widening is given by the hierarchy in Figure 8.2(a) and are used to preserve the information. In widening hierarchy, any lower type can be widened into a higher type like a `byte` can be widened to a `short` or to an `int` or to a `float`, but a `short` cannot be widened to a `char`.

The narrowing conversions, on the other hand, may result in loss of information. The rules used for narrowing is given by the hierarchy in Figure 8.2(b), in which a type x can be narrowed to type y if and only if there exists a path from x to y . Note that `char`, `short`, and `byte` are pairwise convertible to each other.

Multiple-Choice Questions

- Which of the following is true for type system?
 - It is a tractable syntactic framework.
 - It uses logical rules to determine the behavior of a program.
 - It guarantees that only value specific operations are allowed.
 - All of these
- A type system can be _____ type system or _____ type system.
 - Basic, constructed
 - Static, dynamic
 - Simple, compound
 - None of these

3. Which of the following is true for type checking?
 - (a) It ensures type correctness.
 - (b) It can only be done at compile time.
 - (c) It can only be done at runtime.
 - (d) All of these
4. A type checking is called strongly typed if _____.
 - (a) It is performed at runtime.
 - (b) It is performed at compile time.
 - (c) The type checking rules are performed strongly.
 - (d) Both (a) and (b)
5. In type synthesis, the names must be _____.
 - (a) Declared after their use
 - (b) Declared before their use
 - (c) Need not be declared
 - (d) Depends on the parent expressions
6. Why type expressions are used?
 - (a) To free our program from errors
 - (b) To represent structure of types
 - (c) To represent textual representation for types
 - (d) Both (b) and (c)
7. Which of the following is not true for static type checking?
 - (a) It is performed at compile time.
 - (b) It catches errors at compile time.
 - (c) Most of the properties are verified at compile time.
 - (d) It provides flexibility of performing type conversions at runtime.
8. A strong type checking ensures that _____.
 - (a) No type errors can occur at compile time.
 - (b) No type errors can occur at runtime.
 - (c) Both (a) and (b)
 - (d) None of these
9. Implicit type checking is also known as _____.

(a) Casts	(b) Explicit conversion
(c) Manual conversion	(d) Coercion

Answers

1. (a) 2. (a) 3. (a) 4. (c) 5. (b) 6. (d) 7. (d) 8. (b) 9. (d)

Runtime Administration

1. Define runtime environment. What are the issues in runtime environment?

Ans: The source language definition contains various abstractions such as names, data types, scopes, bindings, operators, parameters, procedures, and flow of control constructs. These abstractions must be implemented by the compiler. To implement these abstractions on target machine, compiler needs to cooperate with the operating system and other system software. For the successful execution of the program, compiler needs to create and manage a **runtime environment**, which broadly describes all the runtime settings for the execution of programs.

In case of compilation of a program, the runtime environment is indirectly controlled by generating the code to maintain it. However, in case of interpretation of the program, the runtime environment is directly maintained by the data structures of the interpreter.

The runtime environment deals with several issues which are as follows:

- ❑ The allocation and layout of storage locations for the objects used in the source program.
- ❑ The mechanisms for accessing the variables used by the target program.
- ❑ The linkages among procedures.
- ❑ The parameter passing mechanism.
- ❑ The interface to input/output devices, operating systems and other programs.

2. What are the important elements in runtime environment?

Ans: The important elements that constitute a runtime environment for a program are as follows:

- ❑ **Memory organization:** During execution, a program requires certain amount of memory for storing the local and global variables, source code, certain data structures, and so on. The way memory is organized for storing these elements is an important characteristic of runtime environment. Different programming languages support different memory organization schemes. For example, C++ supports the use of pointers and dynamic memory with the help of `new()` and `delete()` functions, whereas FORTRAN 77 does not support pointers and usage of dynamic memory.
- ❑ **Activation records:** The execution of a procedure in a program is known as the **activation** of the procedure. The activation of procedures or functions is managed with the help of a contiguous

block of memory known as **activation record**. Activation record can be created statically or dynamically. Statically, a single activation record can be constructed, which is common for any number of activations. Dynamically, number of activation records can be constructed, one for each activation. The activation record contains the memory for all the local variables of the procedure, depending on the way by which activation record is created, the target code has to be generated accordingly to access the local variables.

- ❑ **Procedure calling and return sequence:** Whenever a procedure is invoked or called, certain sequence of operations need to be performed, which include evaluation of function arguments, storing it at a specified memory location, transferring the control to the called procedure, etc. This sequence of operations is known as **calling sequence** and **procedure calling**. Similarly, when the activated procedure terminates, some other operations need to be performed such as fetching the return value from a specified memory location, transferring the control back to the calling procedure, etc. This sequence of operations is known as **return sequence**. The calling sequence and return sequence differ from one language to another, and in some cases even from one compiler to another for the same language.
- ❑ **Parameter passing:** The functions used in a program may accept one or more parameters. The values of these parameters may or may not be modified inside the function definition. Moreover, the modified values may or may not be reflected in the calling procedure depending on the language used. In some languages like PASCAL and C++, some rules are specified which determine whether the modified value should be reflected in the calling procedure. In certain languages like FORTRAN77 the modified values are always reflected in the calling procedure. There are several techniques by which parameters can be passed to functions. Depending on the parameter passing technique used, the target code has to be generated.

3. Give subdivision of runtime memory.

Or

What is storage organization?

Or

Explain stack allocation and heap allocation?

Or

What is dynamic allocation? Explain the techniques used for dynamic allocation (stack and heap allocation).

Ans: The target program (already compiled) is executed in the runtime environment within its own logical address space known as **runtime memory**, which has a storage location for each program value. The compiler, operating system, and the target machine share the organization and management of this logical address. The runtime representation of the target program in the logical address space comprises data and program areas as shown in Figure 9.1. These areas consist of the following information:

- ❑ The generated target code
- ❑ Data objects
- ❑ Information to keep track of procedure activations.

Since the size of the target code is fixed at compile time, it can be placed in a statically determined area named *Code* (see Figure 9.1), which is usually placed in the low end of memory. Similarly, the memory occupied by some program data objects such as global constants can also be determined at

compile time. Hence, the compiler can place them in another statically determined area of memory named *Static*. The main reason behind the static allocation of as many data objects as possible is that the compiler could compile the addresses of these objects into the target code. For example, all the data objects in FORTRAN are statically allocated.

The other two areas, namely, *Stack* and *Heap* are used to maximize the utilization of space of runtime. The size of these areas is not fixed, that is, as the program executes their size can change. Hence, these areas are dynamic in nature.

Stack allocation: The stack (also known as **control stack** or **runtime stack**) is used to store activation records that are generated during procedure calls. Whenever a procedure is invoked, the activation record corresponds to that procedure is pushed onto the stack and all local items of the procedure are stored in the activation record. When the execution of procedure is completed, the corresponding activation record is popped from the stack and the values of locals are deleted.

The stack is used to manage and allocate storage for the active procedure such that

- ❑ On the occurrence of a procedure call, the execution of the calling procedure is interrupted, and the activation record for the called procedure is constructed. This activation record stores the information about the status of the machine.
- ❑ On receiving control from the procedure call, the values in the relevant registers are restored and the suspended activation of the calling procedure is resumed, and then the program counter is updated to the point immediately after the call. The stack area of runtime storage is used to store all this information.
- ❑ Some data objects which are contained in this activation and their relevant information are also stored in the stack.
- ❑ The size of the stack is not fixed. It can be increased or decreased according to the requirement during program execution.

Runtime stack is used in C and Pascal.

Heap allocation: The main limitation of stack area is that it is not possible to retain the values of non-local variables even after the activation record. This is because of last-in-first-out nature of stack allocation. To retain the values of such local variables, heap allocation is used. The heap allocates a contiguous memory locations as and when required for storing the activation records and other data elements. When the activation ends, the memory is deallocated, and this free space can be further used by the heap manager. The heap management can be made efficient by creating a linked list of free blocks. Whenever some memory is deallocated, the free block is appended in the linked list, and when memory needs to be allocated, the most suitable (best-fit) memory block is used for allocation. The heap manager dynamically allocates the memory, which results into a runtime overhead of taking care of defragmentation and garbage collection. The garbage collection enables the runtime system to automatically detect unused data elements and reuse their storage.

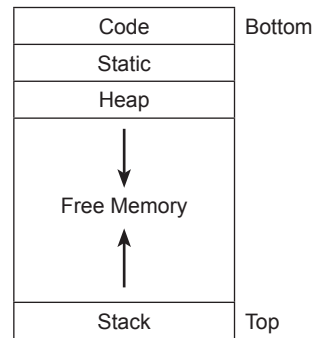


Figure 9.1 Subdivision of Runtime Memory

4. Explain static allocation. What are its limitations?

Ans: An allocation is said to be static if all data objects are stored at compile time. It has the following properties:

- The binding of names is performed during compilation and no runtime support package is required.
- The binding remains same at runtime as well as compile time.
- Each time a procedure is invoked, the names are bounded to the same storage locations. The values of local variables remain unchanged before and after the transfer of controls.
- The storage requirement is determined by the type of a name.

Limitations of static allocation are given as follows:

- The information like size of data objects and constraints on their memory position needs to be present during compilation.
- Static allocation does not support any dynamic data structures, because there is no mechanism to support run-time storage allocation.
- Since all the activations of a given procedure use the same bindings for local names, recursion is not possible in static allocation.

5. Explain in brief about control stack.

Ans: A stack representing procedure calls, return, and flow of control is called a **control stack** or **runtime stack**. Control stack manages and keeps track of the activations that are currently in progress. When the activation begins, the corresponding activation node is pushed onto the stack and popped out when the activation ends. The control stack can be nested as the procedure calls or activations nest in time such that if p calls q , then the activation of q is nested within the activation of p .

6. Define activation tree.

Ans: During the execution of program, activation of the procedures can be represented by a tree known as **activation tree**. It is used to depict the flow of control between the activations. Activations are represented by the nodes in activation tree where each node corresponds to one activation, and the root node represents the activation of the `main` procedure that initiates the program execution. Figure 9.2 shows that the `main()` activates two procedures P_1 & P_2 . The activations of procedures P_1 & P_2 are represented in the order in which they are called, that is, from left to right. It is important to note that the left child node must finish its execution before the activation of right node can begin. The activation of P_2 further activates two procedures P_3 and P_4 . The flow of control between the activations can be depicted by performing a depth first traversal of the activation tree. We start with the root of the tree. Each node is visited before its child nodes are visited and the child nodes are visited from left to right. When all the child nodes of a particular node have been visited, we can say that the procedure activation corresponding to a node is completed.

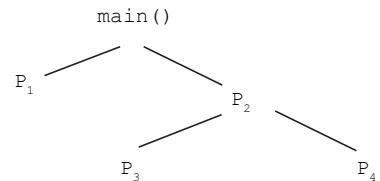


Figure 9.2 Activation Tree

7. Discuss in detail about activation records.

Ans: The **activation record** is a block of memory on the control stack used to manage information for every single execution of a procedure. Each activation has its own activation record with the root of activation tree at the bottom. The path from one activation to another in the activation tree is determined by the corresponding sequence of activation records on the control stack.

Different languages have different activation record contents. In FORTRAN, the activation records are stored in the static data area while in C and Pascal, the activation records are stored in stack area. The contents of activation records are shown in Figure 9.3.

The various fields of activation record are as follows:

- ❑ **Temporaries:** The temporaries are used to store intermediate results that are generated during the evaluation of an expression and cannot be held in registers.
- ❑ **Local data:** This field contains local data like local variables, which are local to the execution of a procedure stored in the activation record.
- ❑ **Saved machine status:** This field contains the information regarding the state of a machine just before the procedure is called. This information consists of the machine register contents and the return address (program counter value).
- ❑ **Access link:** It is an optional field and also called **static link** field. It is a link to non-local data in some other activation record which is needed by the called procedure.
- ❑ **Control link:** It is also an optional field and is called **dynamic link** field. It points to the activation record of the calling procedure.
- ❑ **Returned value:** It is also an optional field. It is not necessary that all the procedures return a value, but if the procedure does, then for better efficiency this field stores the return value of the called procedure.
- ❑ **Actual parameters:** This field contains the information about actual parameters which are used by the calling procedure.

Actual parameters
Returned values
Control link (Dynamic link)
Access link (Static link)
Saved machine status
Local data (variables)
Temporaries

Figure 9.3 Activation Record Model

8. Explain register allocation.

Ans: On the target machine, registers are the fastest for the computation as fetching the data from the registers is easy and efficient as compared to fetching it from the memory. So the instructions that involve the use of registers are much smaller and faster than those using memory operands. The main problem with the usage of the register is that the system has limited number of registers which are not enough to hold all the variables. Thus, an efficient utilization of registers is very important to generate a good code. The problem of using registers is divided into two sub problems:

- ❑ **Register allocation:** It includes selecting a set of variables to be stored within the registers during program execution.
- ❑ **Register assignment:** It includes selecting a specific register to store a variable.

The purpose of register allocation is to map a large number of variables into a few numbers of registers, which may result in sharing of single register by several variables. However, since two variables in use cannot be kept in the same register at the same time, therefore, the variables that cannot be assigned to any register must be kept in the memory.

Register allocation can be done either in intermediate language or in machine language. If register allocation is done during intermediate language, then the same register allocator can be used for several target machines. Machine languages, on the other hand, initially use symbolic names for registers, and register allocation turns these symbolic names into register numbers.

It is really difficult to find an optimal assignment of registers. Mathematically, the problem to find a suitable assignment of registers can be considered as a NP-Complete problem. Sometimes, the target machine's operating system or hardware uses certain register-usage convention to be observed, which makes the assignment of registers more difficult.

For example, in case of integer division and integer multiplication, some machines use even/odd register pairs to store operands and the results. The general form of a multiplication instruction is as follows:

M a, b

Here, operand a is a multiplicand, and is in the odd register of an even/odd register pair and b is the multiplier, and it can be stored anywhere. After multiplication, the entire even/odd register pair is occupied by the product.

The division instruction is written as

D a, b

Here, dividend a is stored in the even register of an even/odd register pair and the divisor b can be stored anywhere. After division, remainder is stored in the even register and quotient is stored in the odd register.

Now, consider the two three-address code sequences given in Figure 9.4(a) and (b).

These three-address code sequences are almost same; the only difference is the operator in the second statement. The assembly code sequences for these three-address code sequences are given in Figure 9.5(a) and (b).

Here, L, ST, S, M, and D stand for load, store, subtract, multiply, and divide respectively. R_0 and R_1 are machine registers and SRDA stands for Shift-Right-Double-Arithmetic. SRDA $R_0, 32$ shifts the dividend into R_1 and clears R_0 to make all bits equal to its sign bit.

$x = a - b$	$x = a - b$
$x = x * c$	$x = x - c$
$x = x/d$	$x = x/d$
(a)	(b)

Figure 9.4 Two Three-address Code Sequences

L	R_1, a	L	R_0, a
S	R_1, b	S	R_0, b
M	R_0, c	S	R_0, c
D	R_0, d	SRDA	$R_0, 32$
ST	R_1, x	D	R_0, d
		ST	R_1, x
(a)		(b)	

Figure 9.5 Assembly Code (Machine Code) Sequences

9. Explain the various parameter passing mechanisms of a high-level language.

Or

What are the various ways to pass parameters in a function?

Ans: When one procedure calls another, the communication between the procedures occurs through non-local names and through parameters of the called procedure. All the programming languages have two types of parameters, namely, *actual parameters* and *formal parameters*. The **actual parameters** are those parameters which are used in the call of a procedure; however, **formal parameters** are those which are used during the procedure definition. There are various parameter passing methods but most of the recent programming languages use *call by value* or *call by reference* or both. However, some older programming languages also use another method *call by name*.

- **Call by value:** It is the simplest and most commonly used method of parameter passing. The actual parameters are evaluated (if expression) or copied (if variable) and then their r-values are passed to the called procedure. **r-value** refers to the value contained in the storage. The values of actual parameters are placed in the locations which belong to the corresponding formal parameters of the called procedure. Since the formal and actual parameters are stored in different memory locations, and formal parameters are local to the called procedure, the changes made in the values of formal parameters are not reflected in the actual parameters. The languages C, C++, Java, and many more use call by value method for passing parameters to the procedures.
- **Call by reference:** In call by reference method, parameters are passed by reference (also known as **call by address** or **call by location**). The caller passes a pointer to the called procedure, which points to the storage address of each actual parameter. If the actual parameter is a name or an expression having an l-value, then the l-value itself is passed (here, l-value represents the address of the actual parameter). However, if the actual parameter is an expression like $a + b$ or 2 , having

no l-value, then that expression is calculated in a new location, and the address of that new location is passed. Thus, the changes made in the calling procedure are reflected in the called procedure.

- **Call by name:** It is a traditional approach and was used in early programming languages, such as ALGOL 60. In this approach, the procedure is considered as a macro, and the body of the procedure is substituted for the call in the caller and the formals are literally substituted by the actual parameters. This literal substitution is called **macro expansion** or **in-line expansion**. The names of the calling procedure are kept distinct from the local names of the called procedure. That is, each local name of the called procedure is systematically renamed into a distinct new name before the macro expansion is done. If necessary, the actual parameters are surrounded by parentheses to maintain their integrity.

10. What is the output of this program, if compiler uses following parameter passing methods?

(1) Call by value (2) Call by reference (3) Call by name

The program is given as:

```
void main (void)
{
    int a, b;
    void A(int, int, int);
    a = 2, b = 3;
    A(a + b, a, a);
    printf ("%d", a);
}

void A (int x, int y, int z)
{
    y = y + 1;
    z = z + x;
}
```

Ans: Call by value: In call by value, the actual values are passed. The values $a = 2$ and $b = 3$ are passed to the function A as follows:

$A(2 + 3, 2, 2);$

The value of a is printed as 2, because the updated value is not reflected in `main()`.

Call by reference: In call by reference, both formal parameters y and z have the same reference that is, a. Thus, in function A the following values are passed.

$x = 5$
 $y = 2$
 $z = 2$

After the execution of $y = y + 1$, the value of y becomes

$y = 2 + 1 = 3$

Since y and z are referring to the same memory location, z also becomes 3. Now after the execution of statement $z = z + x$, the value of z becomes

$z = 3 + 5 = 8$

When control returns to `main()`, the value of `a` will now become 8. Hence, output will be 8.

Call by name: In this method, the procedure is treated as macro. So, after the execution of the function

```
x = 5
y = y + 1 = 2 + 1 = 3
z = z + x = 2 + 5 = 7
```

When control returns to `main()`, the value of `a` becomes 7. Hence, output will be 7.

Multiple-Choice Questions

- What are the issues that the runtime environment deals with?
 - The linkages among procedures
 - The parameter passing mechanism
 - Both (a) and (b)
 - None of these
- The elements of runtime environment include _____.
 - Memory organization
 - Activation records
 - Procedure calling, return sequences, and parameter passing
 - All of these
- Which of the following area in the memory is used to store activation records that are generated during procedure calls?
 - Heap
 - Runtime stack
 - Both (a) and (b)
 - None of these
- _____ are used to depict the flow of control between the activations of procedures.
 - Binary trees
 - Data flow diagrams
 - Activation trees
 - Transition diagram
- The _____ is a block of memory on the control stack used to manage information for every single execution of a procedure.
 - Procedure control block
 - Activation record
 - Activation tree
 - None of these
- _____ is the process of selecting a set of variables that will reside in CPU registers.
 - Register allocation
 - Register assignment
 - Instruction selection
 - Variable selection

7. Which of the following is the parameter passing mechanism of a high-level language?
- (a) Call by value
 - (b) Call by reference
 - (c) Both (a) and (b)
 - (d) None of these

Answers

1. (c) 2. (d) 3. (b) 4. (c) 5. (b) 6. (a) 7. (c)